Why Cache and Where

keep hot data close, and know what not to cache

Phase 3 videos on YouTube

> **In one line:** a cache is a fast copy of data that is expensive to fetch or compute; it works because a small hot working set gets most requests, the hit ratio decides how much load it removes, caching can happen at every layer from browser to database, and some data (strictly fresh, rarely reused, one-off personal, sensitive) shouldn't be cached.

1 The same query, thousands of times a second

Every product page on your shop runs the same database query, thousands of times a second, for data that changes once a day. The database sits at 90% CPU and pages take half a second. What if most of those requests never reached the database at all? That's what a **cache** does. Welcome to the caching section of Phase 3.

2 What a cache is, and why it works

A cache is a **fast copy** of data that's expensive to fetch or compute. Reading from memory takes well under a microsecond; a database query takes milliseconds; a call to another service even more (Phase 2, estimation lesson 2).

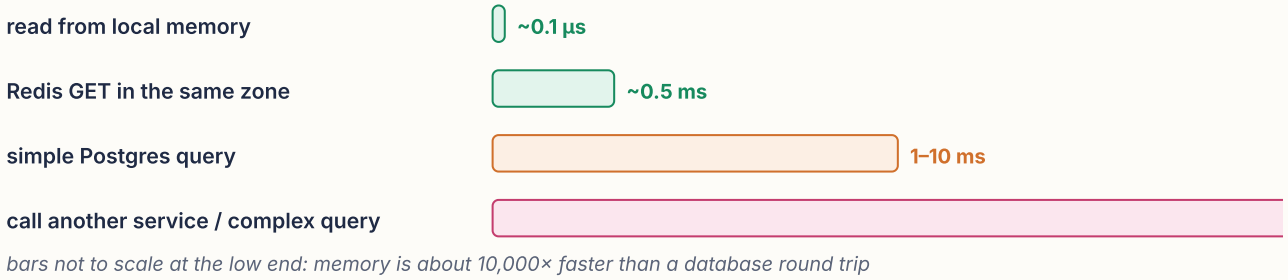


Figure 1. Where the time goes: memory and Redis are orders of magnitude faster than a database or service call.

Caching works because of **locality**: a small set of hot data, the **working set**, gets most of the requests. The top 1% of products might get half the views. Keep that set close, and most reads become fast.

3 The number that matters: hit ratio

The **hit ratio** is the share of requests served from the cache. It matters more than it seems: at 90%, one in ten requests reaches the database; at 99%, only one in a hundred does, **ten times less load**. Every miss also costs extra time: the cache check, then the database trip.

```
average latency ≈ hit ratio × cache time + (1 − hit ratio) × (cache time
                                + database time)
```

With a 1 ms cache and a 20 ms database: 90% → about 3 ms on average; 99% → about 1.2 ms.

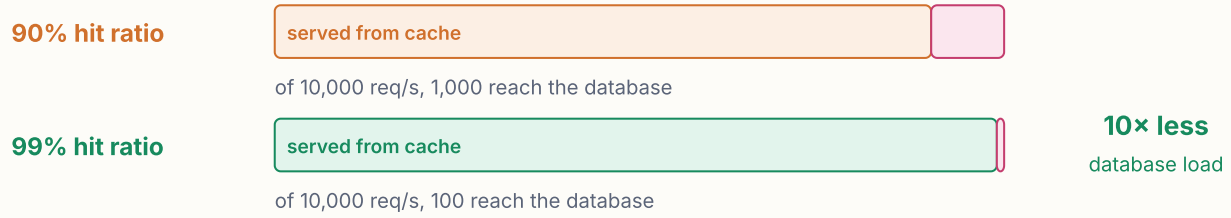


Figure 2. Going from a 90% to a 99% hit ratio cuts database traffic by ten times.

4 Where caching can happen

Along one request there are many places to cache:

- **Browser or mobile app**, controlled by HTTP headers such as `Cache-Control` (lesson 31).
- **CDN** edges cache static files and some API responses near users (Phase 2, lesson 28).
- **API gateway or reverse proxy** can cache whole responses.
- **Application**: objects in process memory, or in a shared cache such as **Redis** or Memcached.
- **Database**: its own buffer pool keeps hot pages in memory.

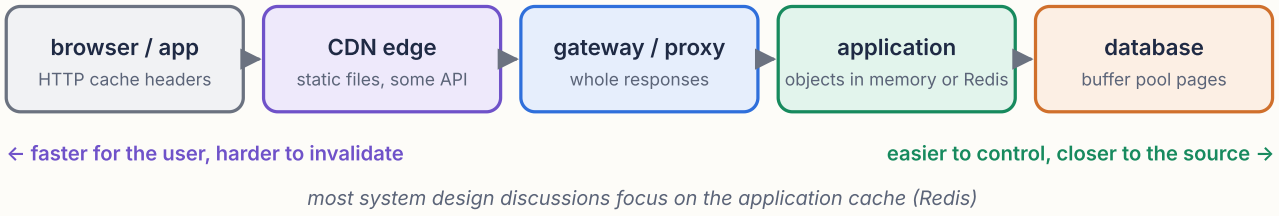


Figure 3. Caching layers along a request, from the user's device to the database.

Each layer trades **freshness for speed**. The closer to the user, the faster, but the harder to invalidate: you can't clear a browser's cache from your side, so keep browser lifetimes short for anything that changes. Application caches in Redis are easiest to control, which is why most system design discussions focus there.

WORKED EXAMPLE: PRODUCT DETAILS IN REDIS

Product details are read constantly but change rarely. The app stores each product in Redis under a key like `product:42` with a 10-minute TTL. With a 95% hit ratio, Postgres handles one twentieth of the reads: CPU drops from 90% to about 20%, and pages respond in a few milliseconds instead of half a second.

5 What not to cache

- **Data that must be exactly current**: account balances, stock levels at checkout. A stale value causes real harm.
- **Data that's rarely reused**: the hit ratio would be low, and entries just take memory.
- **Highly personalised one-off results**: the cache fills with entries read once.
- **Sensitive data**, unless the cache is secured and keys are scoped to the right user, so one user never sees another's data.

WHAT A CACHE COSTS YOU

- **Staleness**: the copy can be out of date (lessons 22 and 23).
- **Another system** to run, monitor and size.
- **New failure modes**: a cold cache after a restart, a stampede on one hot key (lessons 24 to 26).

6 Key terms

Cache: fast copy of expensive data

Hit ratio: share of requests served from the cache

Locality: recent or popular data is likely to be used again

Buffer pool: the database's own in-memory page cache

Hit / miss: found in the cache / not found

Working set: the hot data that gets most requests

TTL: how long an entry stays valid

Staleness: how out of date a cached copy is

7 Interview questions

Q1. Where can caching happen in a request path? BASIC

In the browser or app, the CDN, the API gateway or reverse proxy, the application with in-memory or Redis caches, and the database's own buffer pool.

Q2. What data should not be cached? BASIC

Data that must be strictly fresh, data with low reuse, highly personalised one-off results, and sensitive data without proper isolation.

Q3. Why does a cache help so much? BASIC

Most requests hit a small working set, and memory is thousands of times faster than a database round trip, so serving the hot set from memory removes most of the load and latency.

Q4. Your hit ratio is 80%. How much would reaching 98% reduce database load? SCENARIO

Misses fall from 20% to 2% of requests: ten times fewer database reads.

Q5. Should you cache a user's account balance? SCENARIO

Not for decisions like payments or transfers, which must read the source of truth. For a display-only view you might cache it for seconds, invalidated on every transaction.

Q6. Why can a cache make an outage worse? TRICKY

The database is sized for the cache's hit ratio. If the cache fails or restarts cold, all traffic hits the database at once and can overwhelm it; plan for that with warm-up, rate limiting and request coalescing.

Next: SD-F-002 · Cache-Aside (Lazy Loading) backendroadmap.com